Министерство образования

Дисциплина «Основы разработки безопасных цифровых систем»

Лабораторная работа № 3
«Реализация спроектированной системы»

Выполнил: Го Сяоюй

Группа: 5140901/61501

Вариант 2 — «Журнал для оценки обучающихся»

Оценка: _____

Реестр изменений относительно ЛР № 2

Ниже зафиксировано всё, что отличается реализованной системой от проектного решения в отчёте по ЛР № 2. Расхождений, влияющих на контракты, нет.

#	**Как было спроектировано в ЛР № 2**	**Как реализовано в ЛР № 3**	**Причина**
1	Интеграция с языковой моделью: отдельный контейнер и внешний провайдер	Эндпоинт `POST /retake-requests/{id}/check` оставлен заглушкой и возвращает `501 Not Implemented`	Явное исключение ЛР № 3, п. 3: интеграция с LLM не реализуется; допустимо оставить заглушку 501
2	Клиентское приложение (тип не уточнялся)	Одностраничный веб-клиент `web/index.html`	Допустимо ЛР № 3, Задание 6: если проектировалось мобильное приложение, допускается веб-клиент
3	Отправка письма для восстановления доступа	Письма уходят в локальный почтовый стаб **Mailpit** (порт 8025), без реального SMTP	Разрешено ЛР № 3, п. 3: коммерческие сервисы заменяются локальной заглушкой при сохранении контракта
4	Брокер (RabbitMQ/Kafka) не проектировался; межсервисное взаимодействие — только БД	Реализованы PostgreSQL (основная БД) и Redis (allowlist токенов + одноразовые коды сброса пароля). Асинхронных очередей нет	По ЛР № 3, Задание 5: если в проекте один сервис и БД — задание выполнено автоматически

Все остальные проектные решения перенесены в код без изменений: Argon2id + соль + pepper, JWT (HS256) с отзывом через Redis, отдельная роль `app_api` в БД, оптимистичная блокировка по колонке `version`, объектная авторизация с возвратом `404` на чужой объект, нейтральный `500` без стека и SQL.

1. Цель и что сделано

Цель работы — реализовать и развернуть локально систему «Журнал для оценки обучающихся», спроектированную во второй работе. Контракты ЛР № 2 стали обработчиками, DDL — реальной базой данных, блок решений по безопасности — проверками в коде.

Выполнены все задания ЛР № 3:

1. **Проект и окружение**: git-репозиторий, структура каталогов, `docker-compose.yml`, поднимающий систему одной командой, `.env.example`, README с инструкциями.
2. **База данных**: PostgreSQL 16, схема из `db/01_schema.sql` разворачивается автоматически; приложение под отдельной ролью `app_api` без прав DDL; тестовые данные: два студента, преподаватель, сотрудник, курс CS-101 и оценки.
3. **Аутентификация и токены**: регистрация/вход, Argon2id + соль + pepper, JWT (HS256) с отзывом через Redis, восстановление доступа одноразовым кодом в Mailpit.
4. **Эндпоинты по контракту**: все операции ЛР № 2 (кроме LLM) реализованы; объектная авторизация, оптимистичная блокировка, аудит, параметризованные запросы, нейтральные 500.
5. **Межсервисное взаимодействие**: PostgreSQL + Redis, заявленные в проекте; каждое взаимодействие идёт по контракту.
6. **Клиентское приложение**: веб-клиент для студента (просмотр своих оценок, заявка на пересдачу) и для сотрудника/преподавателя (таблица оценок, изменение значения, подтверждение).
7. **Проверка и карта кода**: коллекция запросов в `requests/`; три проверочных запроса (чужой объект, лишние поля, привилегированная операция) задокументированы в `docs/проверочные_запросы.md`.

2. Технологический стек

Компонент	Технология
Основной сервис (API)	Python 3.12, FastAPI, Uvicorn
База данных	PostgreSQL 16, asyncpg
Кэш / сессии / allowlist токенов	Redis 7
Почтовый стаб	Mailpit (SMTP 1025, UI 8025)

Клиент	Статический HTML/JS (одна страница), отдаётся тем же контейнером API
Развёртывание	Docker Compose, 4 контейнера

3. Запуск

```
cd <папка_репозитория>
docker compose up -d
```

После старта доступно:

- Веб-клиент: `http://localhost:8000/`
- Swagger / OpenAPI: `http://localhost:8000/docs`
- Почтовый ящик (Mailpit): `http://localhost:8025`

Остановка: `docker compose down`. Полный сброс базы к исходному состоянию: `docker compose down -v` и повторный `docker compose up -d` — тестовые данные сеются автоматически.

4. Тестовые учётные записи

Роль	**Email**	**Пароль**
Студент	student@test.edu	Student!123
Студент №2	student2@test.edu	Student2!123
Преподаватель	teacher@test.edu	Teacher!123
Сотрудник / администратор	employee@test.edu	Employee!123

Два студента записаны на один курс CS-101 и имеют собственные оценки — это позволяет на защите показать, что один студент не видит оценки другого.

5. Ключевые решения по безопасности

5.1 Пароли

Пароль хэшируется Argon2id со случайной солью и pepper из переменной окружения. Соль хранится в БД рядом с хэшем, pepper — только в окружении, в репозиторий не попадает. При неверном логине и при неверном пароле система отвечает одинаково.

5.2 Токены

JWT (HS256), ключ подписи читается из `JWT_SECRET`. В Redis хранится allowlist выданных токенов; logout и смена пароля немедленно удаляют токен из Redis — просроченный, подделанный и отозванный токены отклоняются с кодом 401.

5.3 Объектная авторизация

Обработчик `GET /grades/{id}` сначала читает запись, затем проверяет `student_id` текущего пользователя. Если чужой — возвращается `404`, а не `403`: нарушитель не узнаёт, существует ли такой объект.

5.4 Лишние поля в теле запроса

Pydantic-модели запросов не содержат системных полей (`owner_id`, `student_id`, `status`, `role`). Клиент не может «выдать» себе роль или чужого владельца. Имя владельца и статус записи вычисляются сервером.

5.5 Оптимистичная блокировка

`PATCH /grades/{id}` принимает текущую версию; обновление идёт с условием `WHERE id=$1 AND version=$2`. Если версия устарела — `409 Conflict`. Это предотвращает гонку одновременных правок.

5.6 Аудит

Каждое изменение оценки и каждое подтверждение сотрудником пишется в таблицу `audit_log` (кто, что, когда, какое значение). Просмотреть журнал может сотрудник.

5.7 Нейтральные ошибки

Непредвиденное исключение перехватывается middleware и возвращает `500 {"detail": "Внутренняя ошибка сервера"}`. Текст исключения, стек вызовов и SQL уходят только в лог контейнера. Все запросы к БД параметризованы.

5.8 Учётная запись приложения в БД

Сервис подключается под ролью `app_api`, а не под `postgres`. Скрипт `db/02_roles.sql` выдаёт только `SELECT/INSERT/UPDATE/DELETE`; `DROP TABLE` и `CREATE TABLE` под этой ролью завершаются ошибкой прав.

6. Три контрольных запроса (Задание 7.2)

6.1 Чужой объект с валидным токеном

```
GET /api/v1/grades/<id_оценки_студента_2>
Authorization: Bearer <токен_студента_1>
→ HTTP 404 {"detail":"Оценка не найдена"}
```

6.2 Лишние поля в теле запроса

```
PATCH /api/v1/grades/<своя_оценка>
{"value":99,"version":1,
 "owner_id":"<uuid_другого>","status":"closed","role":"employee"}
→ отброшено: владелец и статус в БД не изменены
```

6.3 Привилегированная операция токеном студента

```
POST /api/v1/admin/grades/<id>/confirm
Authorization: Bearer <токен_студента>
→ HTTP 403 {"detail":"Недостаточно прав"}
```

7. Структура репозитория

```
project/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
├── db/
│   ├── 01_schema.sql
│   ├── 02_roles.sql
│   └── 03_seed.sql
├── api/                 # FastAPI-сервис
├── web/
│   └── index.html       # веб-клиент
├── requests/             # коллекция запросов
└── docs/                 # отчёт, карта кода, журнал расхождений
```

8. Ссылка на репозиторий

Репозиторий: **https://github.com/gxy542/qingyanyan1**

Ссылка добавляется после push на GitHub/GitLab; перед сдачей проверить, что ссылка открывается без авторизации.